МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ імені Тараса Шевченка
ФАКУЛЬТЕТ ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ
Кафедра програмних систем і технологій

Дисципліна
« Конструювання програмного забезпечення »

Лабораторна робота № 2

Виконав:	Берещук Назар Олександрович	**Перевірив**:	Шабатін П. Є.
Група	ІПЗ-34-10	Дата перевірки	
Форма навчання	денна	Оцінка	
Спеціальність	121		

2026

Варіант 1: Симулятор системи залежностей (Dependency Resolver)

Архітектура проекту:

Клас DependencyGraph відповідає за побудову графа

Клас InputParser відповідає за встановлення залежностей між пакетами

Клас InputReader відповідає за зчитування файлу

Клас Main є точкою входу в програму

Клас PackageId відповідає за збереження даних про пакет

Клас ReportGeneretor відповідає за виведення результату програми

Клас ResolverEngine відповідає за сортування пакетів та встановлення порядку їх встановлення

Клас ValidationEngine відповідає за виявлення циклів

Детально по класам:

Main

При запуску програми передається файл із пакетами, далі він читається за допомогою класу InputReader, та передається в InputParser для встановлення залежностей між пакетами, після цього пошук циклів, якщо немає то відбувається топологічне сортування для встановлення порядку залежностей.

```java
String filePath = args[0];

try
{

    InputReader reader = new InputReader();
    List<String> lines = reader.readFile(filePath);

    InputParser parser = new InputParser();
    DependencyGraph graph = parser.parse(lines);

    ValidationEngine validator = new ValidationEngine();

    if (validator.hasCycle(graph))
    {

        ReportGenerator report = new ReportGenerator();
        report.printCycleError();

    } else
    {

        ResolverEngine resolver = new ResolverEngine();
        List<PackageId> order = resolver.topologicalSort(graph);

        ReportGenerator report = new ReportGenerator();
        report.printInstallOrder(order);
    }

} catch (Exception e)
{
    System.out.println("Error: " + e.getMessage());
}
```

InputReader

Звичайне читання файлу

```java
public List<String> readFile(String path) throws IOException  1 usage  Назар Берещук
{
    return Files.readAllLines(Path.of(path));
}
```

DependencyGraph

Відповідає за додавання пакету залежностей, додавання самого пакета, та повернення інформації про нього і його залежностей

```java
private Map<PackageId, List<PackageId>> graph;  7 usages

public DependencyGraph()  6 usages  Назар Берещук
{
    graph = new HashMap<>();
}

public void addPackage(PackageId pkg)  1 usage  Назар Берещук
{
    graph.putIfAbsent(pkg, new ArrayList<>());
}

public void addDependency(PackageId from, PackageId to)  1 usage  Назар Берещ
{
    graph.putIfAbsent(from, new ArrayList<>());
    graph.putIfAbsent(to, new ArrayList<>());

    graph.get(from).add(to);
}

public List<PackageId> getDependencies(PackageId pkg)  2 usages  Назар Берещук
{
    return graph.getOrDefault(pkg, new ArrayList<>());
}

public Map<PackageId, List<PackageId>> getGraph()  2 usages  Назар Берещук
{
    return graph;
}
```

InputParser

Відповідає за прохід по кожному рядку, якщо рядок починається з пакет, то додаємо його в Map, якщо із залежності, то додаємо певному пакеті залежність

```java
public DependencyGraph parse(List<String> lines)  1 usage  Назар Берещук
{

    DependencyGraph graph = new DependencyGraph();
    PackageId currentPackage = null;

    for (String line : lines)
    {

        line = line.trim();

        if (line.startsWith("package:"))
        {

            String[] parts = line.substring( beginIndex: 8).split( regex: ",");
            String name = parts[0].trim();
            String version = parts[1].trim();

            currentPackage = new PackageId(name, version);
            graph.addPackage(currentPackage);

        } else if (line.startsWith("requires:"))
        {

            String[] parts = line.substring( beginIndex: 9).split( regex: ",");
            String name = parts[0].trim();
            String version = parts[1].trim();

            PackageId dependency = new PackageId(name, version);
            graph.addDependency(currentPackage, dependency);
        }
    }

    return graph;
}
```

Package

Відповідає за збереження даних про пакет, та уникненню дублювання об’єктів

```
private String name;  5 usages
private String version;  5 usages

public PackageId(String name, String version)  5 usages  Назар Берещук
{
    this.name = name;
    this.version = version;
}

public String getName() { return name; }

public String getVersion() { return version; }

@Override  Назар Берещук
public boolean equals(Object o)
{
    if (this == o) return true;
    if (!(o instanceof PackageId)) return false;
    PackageId that = (PackageId) o;
    return name.equals(that.name) && version.equals(that.version);
}

@Override  Назар Берещук
public int hashCode()
{
    return Objects.hash(name, version);
}
```

ValidatonEngine

Виявляє цикли шляхи рекурсіїї. Ідея така, що ми рекурсивно проходимось по пакетам та їх залежностях та додаємо їх в visited та в стек, і тоді, коли в стекі знову зустрінеться пакет, який там був, це означає, що цикл замкнувся і він існує, якщо в стекі не було повторів, то циклів не було знайдено

```
public boolean hasCycle(DependencyGraph graph)  1 usage  Назар Берещук
{

    Set<PackageId> visited = new HashSet<>();
    Set<PackageId> recursionStack = new HashSet<>();

    for (PackageId pkg : graph.getGraph().keySet())
    {
        if (detectCycle(pkg, graph, visited, recursionStack))
        {
            return true;
        }
    }

    return false;
}
```

```java
private boolean detectCycle(  2 usages  Назар Берещук
        PackageId pkg,
        DependencyGraph graph,
        Set<PackageId> visited,
        Set<PackageId> stack)
{

    if (stack.contains(pkg))
    {
        return true;
    }

    if (visited.contains(pkg))
    {
        return false;
    }

    visited.add(pkg);
    stack.add(pkg);

    List<PackageId> dependencies = graph.getDependencies(pkg);

    for (PackageId dep : dependencies)
    {
        if (detectCycle(dep, graph, visited, stack))
        {
            return true;
        }
    }

    stack.remove(pkg);

    return false;
}
```

ResolverEngine

В цьому класі відбувається сортування пакетів, тут схожий алгоритм з пошуком циклів, але ми пакети не додаємо на початку в стек, а додаємо в кінці, коли дійдемо до пакета, який не містить залежностей, таким чином з самого верху буде пакет, який потребує останнім для встановлення

```java
public List<PackageId> topologicalSort(DependencyGraph graph)  1 usage  Назар Берещук
{

    Set<PackageId> visited = new HashSet<>();
    Stack<PackageId> stack = new Stack<>();

    for (PackageId pkg : graph.getGraph().keySet())
    {
        if (!visited.contains(pkg))
        {
            dfs(pkg, graph, visited, stack);
        }
    }

    List<PackageId> result = new ArrayList<>();

    while (!stack.isEmpty())
    {
        result.add(stack.pop());
    }

    return result;
}
```

```java
private void dfs(  2 usages  Назар Берещук
        PackageId pkg,
        DependencyGraph graph,
        Set<PackageId> visited,
        Stack<PackageId> stack)
{

    visited.add(pkg);

    for (PackageId dep : graph.getDependencies(pkg))
    {
        if (!visited.contains(dep))
        {
            dfs(dep, graph, visited, stack);
        }
    }

    stack.push(pkg);
}
```

ReportGenerator

Цей клас відповідає за вивдення результату в консоль, якщо циклів не було знайдено, то друкуємо порядок встановлення, якщо було знайдено, то виводимо повідомлення про цикл

```java
public void printInstallOrder(List<PackageId> order)  1 usage  Назар Берещук
{
    System.out.println("Installation order:");

    for (PackageId pkg : order)
    {
        System.out.println(pkg.getName() + " " + pkg.getVersion());
    }
}

public void printCycleError()  1 usage  Назар Берещук
{
    System.out.println("Error: cyclic dependency detected.");
}
```

Висновок:

Було реалізовано функціонал роботи менеджера пакетів, всі класи, які були на діаграмі класів присутні в програмі, кожен клас відповідає за свою задачу. Програма здатна виконувати пошук циклів та встановлювати порядок встановлення пакетів.